

Bayesian teaching enables probabilistic reasoning in large language models

Received: 8 May 2025

Accepted: 15 December 2025

Published online: 07 January 2026

Linlu Qiu¹✉, Fei Sha², Kelsey Allen^{3,4,5}, Yoon Kim¹, Tal Linzen^{6,7}✉ & Sjoerd van Steenkiste⁶✉

Large language models (LLMs) are increasingly used as agents that interact with users and with the world. To do so successfully, LLMs must construct representations of the world and form probabilistic beliefs about them. To provide personalized recommendations, for example, the LLM needs to infer a user's preferences from their behavior over multiple interactions. The Bayesian inference framework lays out the optimal way for an agent to update its beliefs as it receives new information. We first show that LLMs fall far short of the standard defined by the Bayesian framework. We then show that by teaching LLMs to mimic the predictions of the normative Bayesian model, we can dramatically improve their ability to update their beliefs; this ability generalizes to new tasks. We conclude that LLMs can effectively learn reasoning skills from examples and generalize those skills to new domains.

Humans interact with the world based on our beliefs about it. To effectively support decision making, our beliefs need to correspond to the structure of the world as much as possible; in other words, our beliefs need to be supported by appropriate “world models”^{1–4}. We typically do not have perfect knowledge about the outside world; to the extent that we are uncertain about our environment, our beliefs need to be probabilistic, reflecting this uncertainty. And for these beliefs to remain relevant as the world changes, or as new information about the world becomes available, we need to update our beliefs to reflect the new information. The framework of Bayesian inference describes the normative way in which new information should trigger a change in one's beliefs so as to maximize the effectiveness of these beliefs as a foundation for acting in the world⁵. The Bayesian framework has informed a substantial body of work in cognitive science, which has identified both areas where humans act as the framework predicts, as well as deviations from it^{6–17}.

In the last few years, artificial intelligence systems based on large language models (LLMs) have become dramatically more capable than in the past^{18–22}. Far outgrowing their original motivation—as methods to estimate the probabilities of different word sequences—these systems are now being used for applications where they interact with users and

with the outside world. As with humans, for the LLMs' interactions with users to be effective, the LLMs' beliefs need to reflect their experience with the user and to be continuously updated as more information becomes available. Here, we ask: do LLMs act as if they have probabilistic beliefs that are updated as expected from normative Bayesian inference? To the extent that the LLMs' behavior deviates from the normative Bayesian strategy, how can we minimize these deviations?

We begin to study these questions using a simple controlled setting: a flight recommendation task²³, illustrated in Fig. 1. This task involves multiple rounds of interactions between a simulated user and an LLM, where the LLM is acting as a flight booking assistant. In each round, the assistant is given a small number of flight options and is expected to recommend one of them to the user based on the user's preferences. The user's preferences are not directly communicated to the LLM: it only observes the choices the user makes among the flight options. To make optimal recommendations, then, the LLM must construct an implicit model of the factors that shape the user's preferences, and must reason probabilistically about those factors as it learns about the user's choices across multiple sets of flight options.

We compare the LLMs' behavior to that of a model that follows the normative Bayesian strategy, which we refer to as the *Bayesian*

¹Massachusetts Institute of Technology, Cambridge, MA, USA. ²Meta, Menlo Park, CA, USA. ³Google DeepMind, Mountain View, CA, USA. ⁴University of British Columbia, Vancouver, BC, Canada. ⁵Vector Institute, Toronto, ON, Canada. ⁶Google Research, Mountain View, CA, USA. ⁷New York University, New York, NY, USA. Most of the work was done while at Google Research: Linlu Qiu and Fei Sha. ✉ e-mail: linluqiu@mit.edu; linzen@google.com; svansteenkiste@google.com

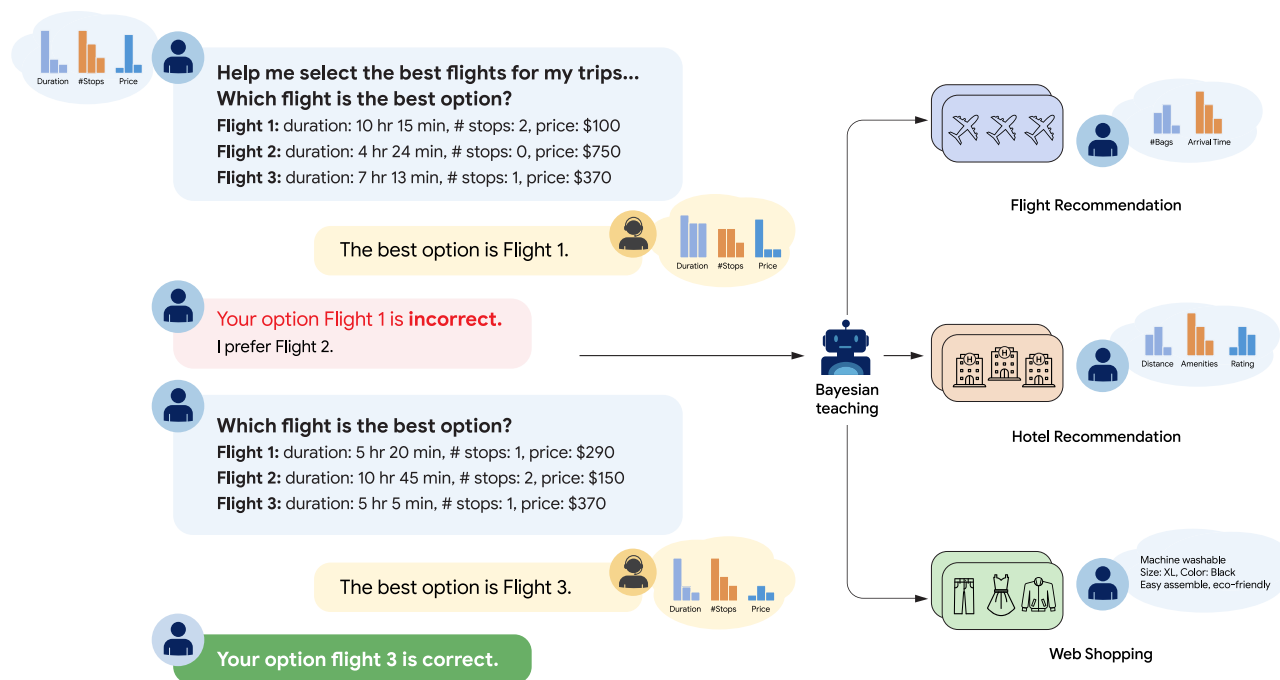


Fig. 1 | Evaluating and improving LLMs' probabilistic belief updates. The flight recommendation task (left) involves multi-round interactions between a user and a flight booking assistant. In each round, the assistant is asked to recommend to the user one of three available flight options. The assistant is then shown the flight that was in fact chosen by the user (based on the user's reward function, which characterizes the user's preferences). To make good recommendations, the assistant

needs to infer the user's preferences from the user's choices. To teach the LLM to reason probabilistically, we fine-tune the LLM on interactions between users and a Bayesian Assistant, which represents the normative way to update beliefs about the user's preferences. We then evaluate the fine-tuned model on the flight recommendation task as well as two new tasks (right).

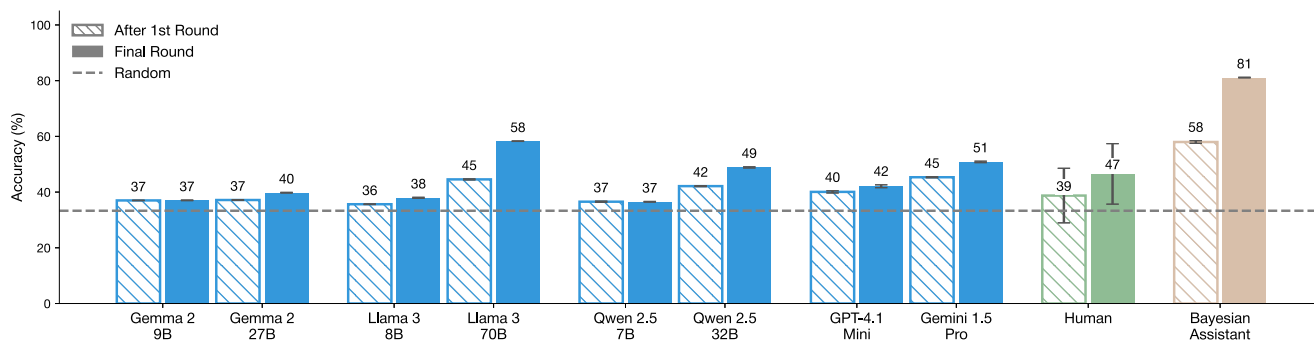


Fig. 2 | LLMs show limited or no improvement over multiple interactions with the user. We show accuracy after the first round and the final (fifth) round. We compare off-the-shelf LLMs from different model families to human participants and the Bayesian Assistant. For human participants, we only evaluate on a subset of 48 out of our 624 simulated users. The LLMs perform considerably worse than the

Bayesian Assistant. Human participants demonstrate a larger improvement than most LLMs as they receive more information, but they still fall short of the accuracy that characterizes the normative Bayesian strategy. For the human study, the error bars show the averaged standard error across participants; for models, they show the standard error across the three sets of interactions with each of the 624 users.

Assistant. This model maintains a probability distribution that reflects its beliefs about the user's preferences, and uses Bayes' rule to update this distribution as new information about the user's choices becomes available. Unlike many real-life scenarios, where it is difficult to specify and implement the Bayesian strategy computationally, in this controlled setting, this strategy can be computed exactly, allowing us to precisely estimate the extent to which LLMs deviate from it.

We use this framework to evaluate a range of LLMs and find that they all perform significantly worse than the normative Bayesian Assistant (Fig. 2). Most importantly, in contrast to the Bayesian Assistant, which gradually improves its recommendations as it receives additional information about the user's choices, LLMs' performance

often plateaus after a single interaction, pointing to a limited ability to adapt to new information.

We then introduce *Bayesian teaching*, a strategy to teach an LLM to approximate Bayesian reasoning. We provide the LLM with examples of interactions between the user and the Bayesian Assistant, and have the LLM mimic those interactions. We find that, by leading the LLMs to gradually adapt to the user over the course of the interactions, this method substantially improves the LLMs' performance on the flight recommendation task. Crucially, teaching the LLMs to mimic the Bayesian Assistant in one task allows them to generalize to other tasks that similarly require making decisions under uncertainty; those include not only different variants of the flight recommendation task, but also a related hotel recommendation task, as well as a web

shopping task with real-world products (Fig. 1), a much more complex task for which it is difficult to specify and implement a fully Bayesian model.

Notably, while the Bayesian Assistant often makes incorrect predictions as it reasons under uncertainty, especially in the early rounds of interaction, we find that it is a more effective teacher than a teacher that directly provides the LLMs with users' choices (which we refer to as an *oracle teacher*); in other words, the Bayesian model's educated guesses make for a stronger learning signal than the correct answers. Overall, we conclude that through observing the Bayesian Assistant perform a particular task, the LLMs are able to approximate transferable probabilistic reasoning skills.

To summarize our contributions: we first identify significant limitations of off-the-shelf LLMs in tasks that require forming and updating probabilistic beliefs. We then demonstrate that, by having the LLMs mimic a normative Bayesian model, we can teach them effectively to approximate probabilistic belief updates, and show that these skills can generalize to new environments. These findings suggest that LLMs can be used in interactive settings where information is provided gradually, including complex application domains where implementing an exact Bayesian model is difficult. More generally, our results highlight a unique strength of deep learning models such as LLMs: they can learn to mimic a symbolic model and generalize its strategy to domains that are too complex to specify in a classic symbolic model.

Results

Evaluating belief updates via flight recommendations

We first describe the simplified flight recommendation task, derived that we use to evaluate the LLMs²³. In this task, we have the LLMs interact with a simulated user for five rounds. In each round, three flight options are presented to both the user and the assistant. Each flight is defined by a departure time, a duration, a number of stops, and a cost (see Fig. 1). Each simulated user is characterized by a set of preferences: for each feature, they can have a strong or weak preference for high or low values of the feature (e.g., they may prefer longer or shorter flights), or no preference regarding this feature. We refer to this set of preferences as the user's *reward function*. We have 624 possible users in total (see "Methods"). These preferences, which determine the flights that the user chooses, are not directly revealed to the assistant. The goal of the assistant is to recommend the flight that matches the user's choice. At the end of each round, the user indicates to the assistant whether or not it chose correctly, and provides it with the correct answer.

After each round, we evaluate the accuracy of the assistant's recommendations for 100 new sets of three flights that differ from the ones on which the assistant has received feedback. We do not provide any feedback to the assistant for these new flight option sets (see Supplementary Fig. A1 for the evaluation workflow).

The Bayesian Assistant

Because the users' preferences are only revealed gradually, through their choices among flight options, we cannot expect the LLMs to reach perfect accuracy immediately after a single round of interaction. As an upper bound on the LLMs' performance, we define a Bayesian Assistant, which implements the strategy that optimally takes into account the evidence about the user's preferences that accumulates over rounds of interaction. This entails maintaining uncertainty about those preferences when the evidence is partial: instead of committing to a single most likely reward function, which could turn out to be incorrect in future rounds, the assistant maintains a probability distribution over possible reward functions. After each round, the Bayesian Assistant updates its distribution over reward functions using Bayes' rule: the probability of each reward function after the round (the posterior) is computed based on its probability before the round

(the prior) and whether or not it was compatible with the user's choice (the likelihood). This normative model represents the best performance that we can possibly expect from any system. Because the number of possible reward functions is small, we are able to perform exact Bayesian inference (see "Methods").

This method requires us to define the Bayesian Assistant's initial prior distribution, that is, its probabilistic assumptions about which user preferences are more likely, in advance of any interaction with the user. We use an uninformed prior, where all possible sets of user preferences are equally likely (for experiments with alternative priors, see Supplementary Section C.4).

LLMs show limited evidence of belief updating

The LLMs we evaluate, like most contemporary LLMs, are first trained to predict upcoming words in a large collection of texts ("pre-training"), and are then specialized to follow user instructions provided in natural language ("instruction-tuning")^{24,25}. Most commercially available models are closed-weights: we can query them, but we cannot access their parameters. We evaluate two such closed-weight models, Gemini 1.5 Pro¹⁸ and GPT-4.1 Mini²⁶, which were among the state-of-the-art LLMs at the time of writing²⁷. We also evaluate the following open-weights models: Gemma 2 (9B and 27B parameters)²¹, Llama 3 (8B and 70B parameters)²⁸, and Qwen 2.5 (7B and 32B parameters)²⁹. We chose those models because their performance was quite competitive, and their weights are openly available, which makes it possible to perform fine-tuning (see the next section). We provide these LLMs with English instructions explaining how to act as a flight booking assistant (see Fig. 1 for an example, and Supplementary Table H3 for a detailed interaction).

We show results in Fig. 2. Overall, the accuracy of the LLMs after the five rounds of interaction is considerably lower than that of the Bayesian Assistant, and most of the models show little improvement after the first round of interaction (Fig. 2 shows results after the first and fifth round; for results after each of the five rounds, see Supplementary Fig. G18). For an exploration of how the models' performance varies across users' possible reward functions, see Supplementary Section C.2.

A range of follow-up experiments failed to produce meaningful improvement in the LLMs' behavior (for details, see Supplementary Section B.1). Those include experiments with "chain-of-thought prompting"^{30–32}, that is, instructions that are meant to encourage the LLM to reason more explicitly (Supplementary Fig. B3a); an experiment with alternative, purely numerical representations of the flight options that we hypothesized might be easier for the LLMs to parse than the verbal ones we used for our main experiments (Supplementary Fig. B3b); a setting where we have 30 instead of five rounds of interaction (Supplementary Fig. B3c); and experiments with models that are only pre-trained to predict upcoming words in texts, without subsequent training to follow user instructions (Supplementary Fig. B3f).

We also had human participants act as the assistant to a subset of 48 simulated users (see "Methods" and Supplementary Section E.1 for details). The human participants made recommendations for five rounds and showed a significant improvement between rounds 1 and 5 ($p = 0.002$, logistic mixed-effects model). In terms of accuracy, they perform better than small LLMs and slightly worse than larger LLMs (see Supplementary Fig. G18 for performance over rounds). That being said, like all LLMs, humans also fall substantially short of the accuracy expected from the normative Bayesian strategy.

Teaching LLMs to approximate Bayesian reasoning

We next describe the supervised fine-tuning technique we use to teach the LLM to mimic the normative Bayesian model; we show that this method substantially improves the LLM's ability to update its beliefs correctly.

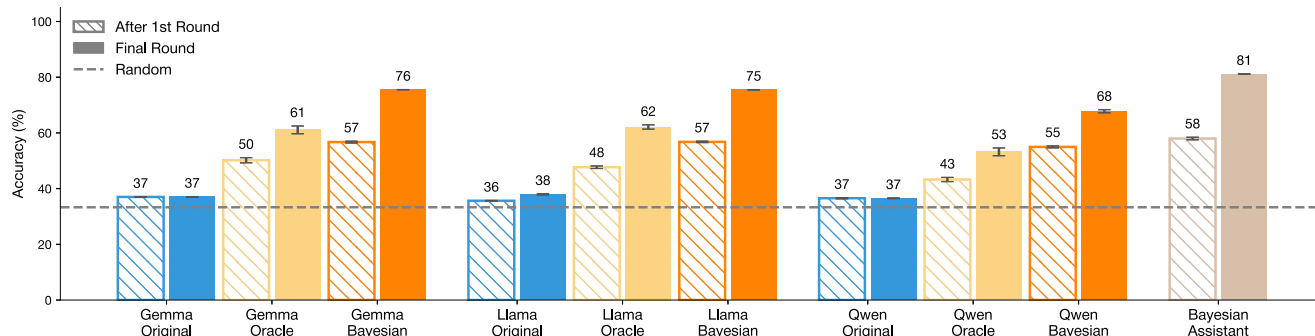


Fig. 3 | Supervised fine-tuning teaches LLMs to approximate probabilistic inference. We show accuracy after the first round and the final (fifth) round across different assistants. We compare the original LLMs, LLMs fine-tuned on user interactions with the Bayesian Assistant, and LLMs fine-tuned on user interactions with an oracle, which always provides the correct answer. Both types of fine-tuning

significantly improve LLMs' performance, and Bayesian teaching is consistently more effectively than oracle teaching. Error bars show the standard error across three random seeds (and three training runs). All results are statistically significant, $p < 0.001$ (see Supplementary Section F).

From a technical perspective, supervised fine-tuning is similar to the method used to train most LLMs in the first place. The model is provided with the first words of a text and is trained to predict the upcoming word. After each example, the LLM's weights are adjusted to increase the likelihood of a correct prediction if the same example is observed again. The main difference is that while in the first phase of training the texts are typically drawn from the Internet or similar resources, in the supervised fine-tuning phase the texts are constructed in a targeted way (automatically or by human writers) so as to teach the LLM particular skills^{24,25}; to improve arithmetic skills, for example, the model may be given the text "the output of $1 + 1 = \dots$ is 2". We apply supervised fine-tuning to the three medium-sized open-weights models (Gemma 2 9B, Llama 3 8B, and Qwen 2.5 7B); we do not attempt to fine-tune the larger models from these families due to computational constraints. We update all of the models' weights in fine-tuning (in Supplementary Section B.2, we show that a different training objective, Direct Preference Optimization³³, produces similar results, as does a computationally cheaper fine-tuning method, LoRA³⁴, which only updates a subset of the model's weights).

We explore two strategies to create supervised fine-tuning data. For both strategies, we construct 10 five-round interactions per user. These interactions follow the same format as described above (Supplementary Table H3). In the first strategy, which we refer to as *oracle teaching*, we provide the LLM with interactions between simulated users and an "oracle" assistant that has perfect knowledge of the user's preferences, and as such always recommends the option that is identical to the user's choices.

The second strategy, which we call *Bayesian teaching*, provides the LLM with interactions between the user and the Bayesian Assistant. In this setting, the assistant will often choose flights that do not match the user's preferred choice, especially in early rounds where there is considerable uncertainty about the user's preferences. We hypothesize that, despite this fact, mimicking the Bayesian Assistant's best guesses would teach the LLM to maintain uncertainty and update its beliefs more effectively than the first strategy, where the LLM is trained on the correct choices. This approach can be seen as a form of distillation, where a model is trained by learning to mimic another system^{35–42}. We use a uniform prior for the Bayesian Assistant that produces the supervised fine-tuning data. Other priors perform similarly (see Supplementary Fig. C10b).

Fine-tuning teaches LLMs to adapt to users

Both supervised fine-tuning strategies, oracle teaching and Bayesian teaching, significantly improve the LLMs' performance on the flight recommendation task (Fig. 3). Crucially, after fine-tuning, the LLMs'

performance gradually improves as more information becomes available; this contrasts with the original LLMs, which plateaued after the first round (see the substantial performance improvement between the first and last round in Fig. 3; for detailed results for each round, see Supplementary Fig. G19). While there is still a performance gap between the fine-tuned LLMs and the normative Bayesian Assistant, this gap is much narrower than for the original LLMs. All three medium-sized LLMs, which before fine-tuning performed worse than either stronger models or our human participants, markedly outperform them after fine-tuning.

We find that Bayesian teaching leads to higher accuracy and less variability across repetitions of the experiment than oracle teaching (Fig. 3). Bayesian teaching also successfully makes the LLM more Bayesian: the Bayesian-tuned LLMs' predictions agree with those of the Bayesian Assistant around 80% of the time, significantly more often than do the predictions of the original LLMs and oracle-tuned LLMs (Fig. 4). In Supplementary Section C.4, we show that the effectiveness of Bayesian teaching cannot be explained by two potential confounds, and conclude that the effectiveness of this method is in fact due to the Bayesian signal it provides.

The amount of information that can be gained from the user's choice for a particular option set varies from one set to another. For example, a choice between two flight options that differ in exactly one feature provides direct evidence for the user's preference for that feature; such a choice could be more informative about the user's preferences than the choice between options that differ along multiple dimensions. We expect a model with more sophisticated probabilistic skills to show greater sensitivity to this factor. Do our fine-tuned models show such sensitivity? Focusing on the Gemma models, we find that Gemma Original does not show sensitivity to option set informativity, but both fine-tuned versions of Gemma do, with Gemma Bayesian displaying considerably more sensitivity than Gemma Oracle (Supplementary Section D).

Can the fine-tuned models accurately verbalize their beliefs? To address this question, we ask the LLMs explicitly for their beliefs about the user's preferences—we have the simulated user ask them, for example, "on a scale of 1 to 5, what is my preference for price?". We then test for the accuracy of these verbalized beliefs by deriving flight recommendations from those beliefs, using the same decision procedure we use with the Bayesian Assistant. We find that this approach generally performs better the approach we have used so far where we directly ask for the LLMs' recommendations; that predictions based on the fine-tuned LLMs' verbalized beliefs are substantially more accurate than those based on the original LLMs' verbalized beliefs; and that the Bayesian-tuned LLMs produce more accurate beliefs than either the original LLMs or oracle-tuned ones (for additional details, see Supplementary Section A).

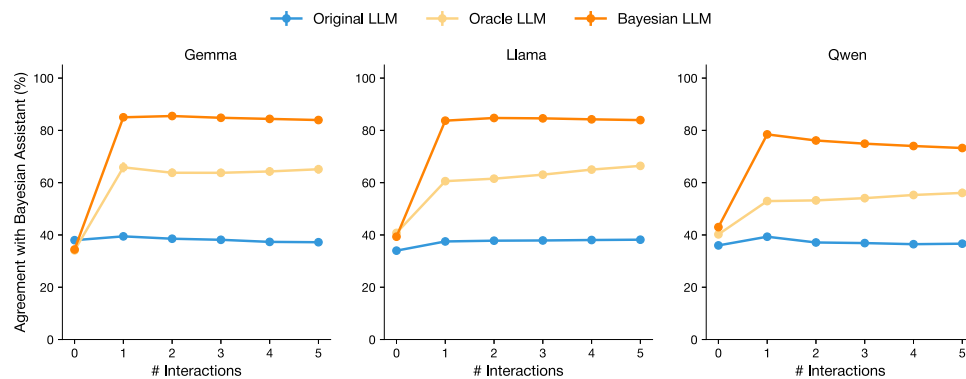


Fig. 4 | Fine-tuned LLMs agree more with the Bayesian Assistant. We show agreement between the LLMs and the Bayesian Assistant, measured by the proportion of trials where the LLMs make the same predictions as the Bayesian Assistant. Fine-tuning on the Bayesian Assistant's predictions makes the LLMs

more Bayesian, with the Bayesian versions of each LLM achieving the highest agreement with the Bayesian Assistant. Error bars (too small to be visible in the plot) show standard errors across three random seeds (and three training runs).

Fine-tuned LLMs generalize to new tasks

As a result of Bayesian teaching, the LLMs demonstrate a greatly improved ability to approximate Bayesian probabilistic inference. Is this ability specific to the particular task the models were trained on, or do the LLMs' probabilistic skills improve more broadly? To answer this question, we evaluate the fine-tuned LLMs on a set of tasks that diverge to different extents from our original flight recommendation task (see the right panel of Fig. 1 for an overview). All tasks require the LLMs to infer the user's preferences from the user's choices over multiple interactions. Overall, as we show in the rest of this section, we find that fine-tuned LLMs show considerable generalization to new settings, and that, as before, Bayesian teaching is more effective than oracle teaching.

We first test the LLMs on variants of the flight recommendation task with different numbers of features: whereas in the interactions provided during fine-tuning, flights were characterized by four features, in this evaluation setting, flights are described by between two and eight features. This requires the LLM to generalize to features that were not included in fine-tuning (e.g., the number of checked bags). In this setting, we find that both types of fine-tuning lead to large improvements in accuracy compared to the original LLMs. We also find that Bayesian teaching is considerably more effective than oracle teaching, as before (Fig. 5a). We note that as the number of features increases, the space of possible reward functions grows exponentially, and the task becomes inherently more difficult, even for the Bayesian Assistant. Despite this fact, for both fine-tuning methods, performance relative to the upper bound defined by the Bayesian Assistant drops off only moderately as the number of features increases.

The generalization experiments we have discussed so far focused on variants of the flight recommendation task. We next evaluate whether the LLMs can generalize the probabilistic skills they acquire through fine-tuning and apply them to other domains. We consider two such domains: hotel recommendations and web shopping. The hotel recommendation task is a synthetic task whose structure is similar to that of the flight recommendation task presented in fine-tuning. Here, each hotel is defined by four features: distance to downtown, price, rating, and amenities (for example, see Supplementary Table H11).

The web shopping task uses real-world products from a simulated environment⁴³, and differs much more substantially from the fine-tuning task than does the hotel recommendation task. It is difficult to construct a Bayesian Assistant for more natural scenarios like the web shopping task, where the space of user preferences is large and hard to specify formally. For this reason, successful transfer from synthetic settings like the flight recommendation task to more natural scenarios represents a particularly important application of Bayesian teaching. In the web shopping task, each user is defined by a set of randomly

sampled goals that characterize the product they are interested in; for example, they might be looking for a shirt that is machine washable, or for a size XL shirt (see Supplementary Table C1 for examples). As in the flight domain, the assistant interacts with the user for multiple rounds. In each round, a set of product options is randomly sampled from the product category (e.g., shirts), and the assistant is asked to recommend the best option. Each product is represented by a short title along with a detailed description (see Supplementary Table H12 for an example). The user provides feedback at the end of each round, indicating whether or not the assistant's recommendation was correct. The user's preferred option is the one with the highest reward, as defined in Yao et al.⁴³. As mentioned above, it is difficult to construct a Bayesian Assistant for this task due to the large space of possible preferences. Instead, as an alternative upper bound on the transfer performance we can expect from the models fine-tuned on the flight recommendation task, we fine-tune LLMs directly on data from the shopping task.

We find that LLMs fine-tuned on the flight recommendation task generalize to both hotel recommendations and web shopping: they perform much better than the original LLMs on those tasks (Fig. 5b, c). Bayesian teaching continues to outperform oracle teaching, though the gap is smaller for web shopping than for hotel recommendations. There remains a gap between the generalization performance of the LLMs fine-tuned on flight recommendations and the upper bound obtained by fine-tuning the LLMs directly on the web shopping interactions (green dashed line in Fig. 5c). Overall, we conclude that fine-tuning, and especially Bayesian teaching, imparts probabilistic skills that transfer substantially beyond the setting used for fine-tuning.

Generalization to interactions with human users

The synthetically generated data we have used so far makes two simplifying assumptions: the simulated users' choices faithfully reflect the reward function that characterizes their preferences, and all reward functions are encountered equally often. In practice, these assumptions may not hold as humans' behavior could occasionally be inconsistent with their preferences, due to inattention or other biases, and some preferences may be more common in the population than others (such as a preference for a lower price). To evaluate the models in a more realistic setting, we recruit human participants to act as users. Each human participant is asked to first state their preferences for each of the flight features, and then select their preferred flight out of three options, for five different sets of options. We collect data from 10 human participants each for 50 lists of flight option sets, for a total of 500 participants (see "Methods").

The performance of both fine-tuned models and the Bayesian Assistant for human users consistently improves over rounds (Fig. 6), and, as was the case for the simulated users, the Bayesian LLMs

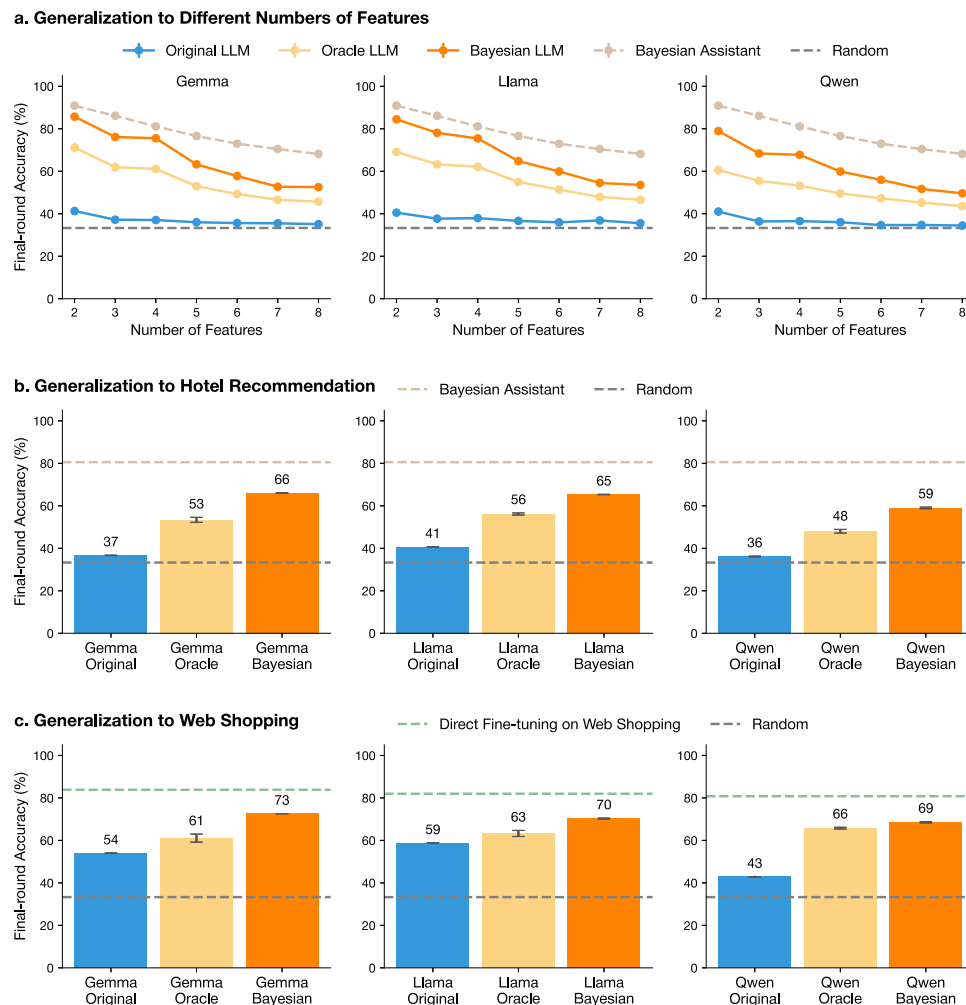


Fig. 5 | Bayesian teaching generalizes outside the task used for fine-tuning. **a** Final-round accuracy in fine-tuned models compared to the original LLM when varying task complexity (here, the number of features is a proxy for task complexity). **b** Final-round accuracy for LLMs on the hotel recommendation task, which was not seen during fine-tuning. We show the normative Bayesian Assistant's performance with brown dashed lines. **c** Final-round accuracy for LLMs on the web

shopping domain, also unseen during fine-tuning. The green dashed line indicates the performance of the LLM when it is fine-tuned directly on web shopping data, such that no domain generalization is necessary. Error bars indicate the standard errors over three training runs (for web shopping) and additionally three random seeds (for flight recommendation and hotel recommendation).

consistently outperform the Oracle LLMs; at least for some model families, the Bayesian LLMs also outperform the original LLMs. This indicates that the Bayesian LLMs generalize to human users from the simulated users on which they were fine-tuned.

All models, including the Bayesian Assistant, show substantially lower performance for humans than they did for simulated users, where accuracy after five rounds approached 80% (Fig. 3). In the Supplementary Section E.2, we show that this is due to the fact that participants' choices are not always consistent with their stated preferences, and as such are impossible to predict with high accuracy (Supplementary Fig. E16a). For the subset of human users whose choices are perfectly consistent with their preferences, the Bayesian LLM performs much better than the original LLM (Supplementary Fig. E15b; see also Supplementary Section C.3, where we study inconsistent simulated users).

Unlike for the simulated users, for human users, the original LLMs perform well even after a single interaction (although, crucially, the original LLMs do not improve over interactions). We attribute the original LLMs' surprisingly strong performance to the fact that human users have generally predictable preferences (e.g., a preference for cheaper flights), such that guesses based on the LLM's priors, without

any adaptation to the individual user, can be quite effective (see Supplementary Figs. E14, E15 for evidence for this hypothesis).

Discussion

To interact with the world successfully, an agent needs to adapt its behavior as it obtains additional information about the statistics of this environment. To evaluate the ability of large language models (LLMs) to do so, we introduced a simple flight recommendation task where, in order to make accurate predictions, the model needs to adapt to a user's preferences over multiple interactions with the user. We tested a range of LLMs and found that they struggle to form and update probabilistic beliefs. We further found that continuing the LLMs' training through exposure to interactions between users and the Bayesian Assistant—a model that implements the normative probabilistic belief update strategy—dramatically improves the LLMs' ability to approximate probabilistic reasoning. Crucially, this improvement did not only hold for the flight recommendation task the LLM was trained on, but also generalized to variants of the flight recommendation task that the LLM had not encountered before, as well as to other tasks. Across the board, this approach, which we refer to as Bayesian teaching, was more effective than a related approach where the LLM is

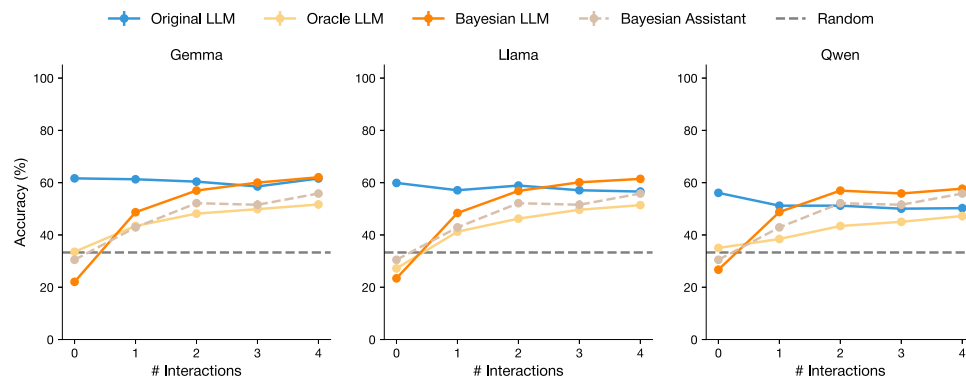


Fig. 6 | Bayesian teaching generalizes to human users. We show accuracy over rounds when the user is a human participant. The original LLMs achieve strong performance but do not show any learning behavior. In contrast, fine-tuned LLMs (with both Bayesian and oracle teachers) improve their performance over rounds,

and the Bayesian LLMs consistently outperform the Oracle LLMs. Error bars show standard errors across four random seeds (and three training runs); the error bars are not visible in the plot because they are very small.

fine-tuned directly on the correct answers, pointing to the effectiveness of the Bayesian training signal.

Our paradigm differs from those used in previous investigations of LLMs' probabilistic reasoning abilities, where LLMs were expected to compute statistics explicitly^{44,45} or provide probability judgments^{46,47}. In our paradigm, probabilistic reasoning is as essential as it is in explicit reasoning tasks, but, crucially, it is implicit in the task. Unlike in some recent studies, where the assistant is expected to ask questions to directly elicit the user's preferences^{23,48–54}, our setup expects the assistant to gradually infer the user's preferences by simply observing the user's choices and to provide recommendations that are increasingly in line with the user's true preferences. Finally, our findings are consistent with those of concurrent work⁵⁵, which also investigates LLMs' ability to infer user preferences from different types of dialogs, including a condition where the user accepts or rejects one or more options provided by the assistant—a setup similar to ours—where the models performed poorly. Compared to this concurrent study, our work analyzes the LLMs' behavior through the lens of Bayesian inference and demonstrates the benefits of mimicking a Bayesian model in fine-tuning compared to a more standard fine-tuning strategy, where the model is always provided with the correct answer (oracle teaching, in the terminology we used in the current paper).

We observed robust generalization from the synthetic flight recommendation task on which the LLMs were fine-tuned to the more natural web shopping task. While performance was even stronger when we fine-tuned the LLM directly on interactions from this task (the green dashed line in Fig. 5c), in practice it may be difficult or expensive to collect such data; our synthetic fine-tuning strategy provides an alternative that improves the LLM's probabilistic reasoning abilities across tasks, without requiring collecting additional data and re-training the model on the new domain.

Our proposal is related to but distinct from approaches that embed an LLM inside a neuro-symbolic framework for probabilistic reasoning^{4,56–61}. In those approaches, the LLM is used to translate between natural language inputs and formal representations, which in turn serve as input to a symbolic model that can update its beliefs according to the Bayesian framework⁴. Indeed, we provide further evidence that hybrid methods can outperform the LLM-only approach in Supplementary Section A, where we describe a variation of our method where we first ask the LLM to verbalize its beliefs about the user's preferences, and then we use an external, symbolic system to make predictions based on these verbalized beliefs. The experiments described in that Supplementary section show that in simple tasks where preferences can be mapped to predictions, such hybrid methods indeed outperform a direct interaction with the LLM. Our

preliminary explorations of this approach can be developed in greater detail in future work.

Besides their superior performance in certain cases, neuro-symbolic methods have the benefit of greater interpretability, and their probabilistic inferences could be more robust. Crucially, however, the utility of such methods is limited to problems whose structure can be made explicit in the symbolic component of the system. By contrast, the method we propose empowers the LLM to approximate probabilistic inference on its own, such that it can apply this skill to domains that are hard to codify explicitly in a symbolic system, domains such as the web shopping task we have examined. This approach leverages LLMs' remarkable ability to generalize to new problems defined using natural language.

Notably, even in cases where the domain is simple enough for a purely symbolic model to be constructed, such models may not be consistently more accurate than LLMs. In our study, we found that while for “well-behaved” simulated users a moderate performance gap persisted between the fine-tuned models and the Bayesian Assistant, for human users, whose choices are not always consistent with their preferences, our Bayesian LLMs were in fact superior to the fully symbolic Bayesian Assistant, demonstrating LLMs' greater robustness to noise compared to symbolic models.

We have argued that through mimicking the Bayesian Assistant, the LLMs learn to perform probabilistic inference, albeit only approximately. This hypothesis may appear to be surprising in light of the fact that LLMs' training objective does not explicitly provide supervision for this skill, and that the transformer architecture does not explicitly track probability distributions: it is trained only to predict the next word produced by the Bayesian Assistant. That being said, there is mounting evidence that in order to predict the next token successfully, LLMs can acquire sophisticated representations that match the structure of the process that generated those tokens. In the case of natural language syntax, for example, the internal representations of LLM trained solely to predict upcoming words have been shown to encode abstract features such as syntactic role and grammatical number^{62–64}. It would be a fruitful direction for future work to determine how probabilistic reasoning is implemented by the LLMs' internal representations, for example, by using techniques such as probes and causal interventions^{65–67} to find internal representations of the model's probability distributions over users' preferences, or using circuit analysis⁶⁸ to explore the computations through which the model updates these distributions.

The success of Bayesian teaching in imparting approximate probabilistic reasoning skills to LLMs opens up a range of questions for future work. Would the benefits of Bayesian teaching extend to

larger models than we were able to fine-tune in this work, or to the recent generation of models that are explicitly trained to reason in words²²? Does the benefit of Bayesian teaching extend to continuous domains and real-world applications beyond the ones we evaluated (for example, interactions whose goal goes beyond shopping)? Could we provide the models with a stronger supervision signal—for example, by instructing them to consider explicit probability distributions, by providing them with explicit supervision on the optimal way to update these distributions (for example, by supervising beliefs as in Supplementary Fig. B4c), or by encouraging them to maintain explicit representations of users such that the probability distributions are consistent across interactions with the same user, through methods such as supervised fine-tuning or reinforcement learning?

The goal of this study was not to replicate human behavior in LLMs, but rather to identify methods that can bring LLMs' probabilistic reasoning skills closer to the normative Bayesian strategy: for most applications, we expect AI assistants to follow normative reasoning standards rather than reproduce human deviations from that standard. That being said, our comparisons between LLMs and humans point to a number of directions for future work. Our participants showed substantial deviations from the normative reasoning strategy, in line with prior work on reasoning biases^{14,16,69,70}. To what extent can people be taught to follow the normative strategy more closely? Can participants' apparent biases be explained as consequences of resource limitations⁷¹? How consistent are participants' choices with their stated preferences? Do people's deviations from the normative strategy align with those of LLMs⁶⁹, and what properties of an LLM lead to closer alignment with humans?

While our findings from our first experiment point to the limitations of particular LLMs, the positive findings of our subsequent fine-tuning experiments can be viewed as a demonstration of the strength of the LLM “post-training” paradigm more generally: by training the LLMs on demonstrations of the normative strategy to perform the task, we were able to improve their performance considerably, suggesting that they learned to approximate the probabilistic reasoning strategy illustrated by the demonstrations. The LLMs were able to generalize this strategy to domains where it is difficult to encode it explicitly in a symbolic model, demonstrating the power of distilling a classic symbolic model into a neural network. We hypothesize that this generalization ability is, in part, responsible for LLMs' remarkable empirical success.

Methods

Simulated users in the flight recommendation task

In each round, we presented a set of k flight options $\mathcal{O} = \{o_1, \dots, o_k\}$ to both the simulated user and the assistant (typically $k = 3$). Each flight has a departure time, a duration, a number of stops, and a cost; these four features are encoded in a vector $\phi(o) \in \mathbb{R}^4$. For each flight option, each feature can take one of 11 values uniformly distributed between 0 and 1, except for the number of stops, which has 3 values. This defines 3×11^3 unique flight options. We converted these four numbers into a textual description illustrated in Fig. 1.

The user's preferences are defined by a reward function θ parameterized by four numbers, which indicate the user's preferences for the aforementioned features. The space Θ of reward functions includes all four-dimensional vectors with the values $\{-1, -0.5, 0, 0.5, 1\}$, where -1 corresponds to a preference for low values of this feature (e.g., short flights) and 1 to a preference for high values (e.g., long flights). Given a set of flight options $\mathcal{O}$, the user computes the reward $r(o; \theta) = \theta^T \phi(o)$ of each flight o , and chooses the flight with the highest reward:

$$o^*(\mathcal{O}, \theta) = \operatorname{argmax}_{o \in \mathcal{O}} r(o; \theta). \quad (1)$$

When there was a tie between multiple options, we randomly selected one of the options that had the highest reward. We excluded the reward function $(0, 0, 0, 0)$, that is, the completely indifferent user. This results in a total of $5^4 - 1 = 624$ possible reward functions, corresponding to 624 simulated users. We note that these simulated users are highly simplified and are not meant to capture the full complexity of humans: humans do not always choose the option that maximizes their utility⁷², and their preferences may evolve over time.

The Bayesian Assistant

Since the space of reward functions is relatively small, we were able to perform exact Bayesian updates. In each round, given options $\mathcal{O}$ and the user's preferred option o^* , the Bayesian Assistant updates its posterior as follows:

$$q_B^{i+1}(\theta | \mathcal{O}^{i+1}, o^{*i+1}) = \frac{p(o^{*i+1} | \theta, \mathcal{O}^{i+1}) q_B^i(\theta)}{p(o^{*i+1} | \mathcal{O}^{i+1})}, \quad (2)$$

where the likelihood function indicates whether the reward function is consistent with the user's choice:

$$p(o^* | \theta, \mathcal{O}) = \mathbb{1}[\max_{o \in \mathcal{O}} r(o; \theta) = o^*]. \quad (3)$$

The Bayesian Assistant then makes flight recommendations based on its reward posterior mean, $\hat{\theta} = \mathbb{E}_{q(\theta)}[\theta]$, following Eq. (1). In most experiments, we used the uniform prior (for experiments with other priors, see Supplementary Fig. C10b).

LLMs

Our main experiments focus on the instruction-tuned versions of open-weights models, including models from the Gemma 2²¹, Llama 3²⁸, and Qwen 2.5²⁹ families. We used Gemma 2 models with 9B parameters (<https://huggingface.co/google/gemma-2-9b-it>) and 27B parameters (<https://huggingface.co/google/gemma-2-27b-it>), Llama 3 models with 8B parameters (<https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct>) and 70B parameters (<https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct>), and Qwen 2.5 models with 7B parameters (<https://huggingface.co/Qwen/Qwen2.5-7B-Instruct>) and 32B parameters (<https://huggingface.co/Qwen/Qwen2.5-32B-Instruct>). We also evaluated Gemini 1.5 Pro¹⁸ and GPT-4.1 Mini²⁶, which can only be accessed through an API, as representatives of stronger models whose weights are not accessible. All of the models we use are based on the Transformer neural network architecture⁷³. We used greedy decoding (temperature of 0) for all experiments.

Generalization tasks

For the variants of the flight recommendation task (see “Fine-tuned LLMs generalize to new tasks”), we varied the number of flight features, ranging from two to eight features. The full flight features include the following features, in addition to the above four: arrival time, layover duration, cancellation policy, and number of bags. As the number of possible reward functions grows exponentially with the number of features, we randomly sampled up to 1000 reward functions (simulated users) for each number of features.

For the hotel recommendation task, the hotel features include distance to downtown, price, rating, and amenities. For each hotel option, the distance to downtown and price take one of 11 values uniformly distributed between 0 and 1, while rating and amenities take one of 5 values uniformly distributed between 0 and 1, resulting in $5 \times 5 \times 11^2$ unique hotel options. We evaluated 624 different simulated users, as in the flight recommendation task.

For the web shopping task, we used real-world products that are publicly available at <https://webshop-pnlp.github.io>. We chose the 100 categories with the most products. Each product is described by a title

and bullet point descriptions, whose length is limited to 800 characters. The reward of a user for a product was calculated based on text-matching heuristics on product attributes and options, following Yao et al.⁴³. For each category, we randomly sampled 10 users, each consisting of five rounds of interactions. Performance was evaluated on 100 held-out option sets within the same category.

To reduce the sensitivity of the results to the specific randomly selected option sets, we averaged all experiments over three random seeds for flight and hotel recommendations, and over all categories for web shopping. In each case, we report the mean and the standard error across runs and evaluation seeds.

LLM fine-tuning

We used the instruction-tuned version of Gemma 2 9B, Llama 3 8B, and Qwen 2.5 7B for all fine-tuning experiments. For each reward function, we generated 10 user–assistant interactions, resulting in $624 \times 10 = 6240$ fine-tuning examples, each with five-round interactions. We experimented with fine-tuning on more examples, but did not observe any significant improvement. The interactions were formatted as shown in Supplementary Table H3.

We used full fine-tuning (i.e., all parameters were updated) with a learning rate of $2e-6$, a batch size of 128, and a maximum sequence length of 2048, for 1 epoch. The models were fine-tuned using the standard language modeling objective, i.e., the cross-entropy loss between the model's predicted token probabilities and the ground-truth tokens in the training data. The loss was only computed on the model's responses. For each setup, we trained three models with different random seeds. We conducted all fine-tuning experiments using $4 \times H100$ GPUs based on the standard recipe (<https://github.com/huggingface/alignment-handbook>). Fine-tuning Gemma 2 9B, Llama 3 8B and Qwen 2.5 7B required about an hour for each model.

Human annotations

We collected two sets of human annotations for the flight recommendation task: one where the annotators act as assistants and one where they act as users. The human annotators were recruited online and paid the market rate of \$12 an hour, as suggested by the Prolific platform⁷⁴ we used to recruit participants. See details in Supplementary Section E.

The annotation setup for the assistant role follows the evaluation setup we used for LLMs. In each round, the annotator was asked to make recommendations from three flight options, with each represented in the same format shown to the LLMs. After making their recommendation, the annotator received feedback indicating whether their choice was correct. They were then directed to a preference questionnaire, where they provided their estimates of the user's preferences for each individual feature (see annotation interface in Supplementary Fig. G17). We sampled 48 reward functions by first grouping them based on the L2 distance between their four-dimensional parameter vector and the origin, then sampling from each group proportionally to its size. We had 15 separate participants provide annotations for each of the 48 simulated users (720 human participants in total).

When the annotator serves in the user role, we first ask them to rate their own preferences for different flight features; this serves as their reward function. Then, the annotator was asked to select their preferred option out of three flight options based on their preferences; this was repeated for five rounds. We constructed 50 such lists of five rounds of flight options, and had 10 annotators produce annotations for each of these 50 lists (500 human participants in total). We then produced three randomly shuffled variants of each of the interactions, for a total of 2000 interactions (500 original interactions and 3×500 shuffled interactions). This ensures that a particular option set is not consistently at a particular point in the interaction (for example,

at the end of the interaction, where the participants may be paying less attention). To ensure quality, we required annotators to think for at least 30 s before making their selection.

Data availability

The flight recommendation data and hotel recommendation data were synthetically generated as described in “Methods”. The web shopping data was adapted from <https://github.com/princeton-nlp/WebShop>. All the training and evaluation data are available at <https://zenodo.org/records/17677329>.

Code availability

The code we used for fine-tuning is publicly available at <https://github.com/huggingface/alignment-handbook>. Detailed instructions for fine-tuning can be found at <https://zenodo.org/records/17677329>.

References

- Johnson-Laird, P. N. Mental models in cognitive science. *Cogn. Sci.* **4**, 71–115 (1980).
- Ha, D. & Schmidhuber, J. Recurrent world models facilitate policy evolution. In *Advances in Neural Information Processing Systems* 31 (2018).
- LeCun, Y. A path towards autonomous machine intelligence. *Open Rev.* **62**, 1–62 (2022).
- Wong, L. et al. From word models to world models: Translating from natural language to the probabilistic language of thought. Preprint at *arXiv* <https://arxiv.org/abs/2306.12672> (2023).
- Chater, N., Tenenbaum, J. B. & Yuille, A. Probabilistic models of cognition: conceptual foundations. *Trends Cogn. Sci.* **10**, 287–291 (2006).
- Griffiths, T. L., Chater, N. & Tenenbaum, J. B. *Bayesian Models of Cognition: Reverse Engineering the Mind*. <https://mitpressbookstore.mit.edu/book/9780262049412> (The MIT Press, Cambridge, MA, 2024).
- Jern, A., Lucas, C. G. & Kemp, C. People learn other people's preferences through inverse decision-making. *Cognition* **168**, 46–64 (2017).
- Tenenbaum, J. B., Kemp, C., Griffiths, T. L. & Goodman, N. D. How to grow a mind: Statistics, structure, and abstraction. *Science* **331**, 1279–1285 (2011).
- Xu, F. & Tenenbaum, J. B. Word learning as Bayesian inference. *Psychol. Rev.* **114**, 245 (2007).
- Baker, C., Saxe, R. & Tenenbaum, J. Bayesian theory of mind: modeling joint belief–desire attribution. In *Proceedings of the Annual Meeting of the Cognitive Science Society*. **33** (2011).
- Tenenbaum, J. B., Griffiths, T. L. & Kemp, C. Theory-based Bayesian models of inductive learning and reasoning. *Trends Cogn. Sci.* **10**, 309–318 (2006).
- Chater, N. & Manning, C. D. Probabilistic models of language processing and acquisition. *Trends Cogn. Sci.* **10**, 335–344 (2006).
- Griffiths, T. L., Steyvers, M. & Tenenbaum, J. B. Topics in semantic association. *Psychol. Rev.* **114**, 211–244 (2007).
- Chaigneau, S., Marchant, N. & Rehder, B. Breaking the chains of independence: a Bayesian uncertainty model of normative violations in human causal probabilistic reasoning. *OSF*, (2025).
- Rehder, B. Beyond Markov: accounting for independence violations in causal reasoning. *Cogn. Psychol.* **103**, 42–84 (2018).
- Rottman, B. M. & Hastie, R. Do people reason rationally about causally related events? Markov violations, weak inferences, and failures of explaining away. *Cogn. Psychol.* **87**, 88–134 (2016).
- Sloman, S. A. & Lagnado, D. Causality in thought. *Annu. Rev. Psychol.* **66**, 223–247 (2015).

18. Gemini Team. Gemini 1.5: unlocking multimodal understanding across millions of tokens of context. Preprint at *arXiv* <https://arxiv.org/abs/2403.05530> (2024).
19. Achiam, J. et al. GPT-4 technical report. Preprint at *arXiv* <https://arxiv.org/abs/2303.08774> (2023).
20. Anthropic. Claude 3, <https://www.anthropic.com/news/claude-3-family> (2024).
21. Gemma Team. Gemma 2: improving open language models at a practical size. Preprint at *arXiv* <https://arxiv.org/abs/2408.00118> (2024b).
22. Guo, D., Yang, D. & Zhang, H. et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature* **645**, 633–638 (2025).
23. Lin, J., Fried, D., Klein, D. & Dragan, A. Inferring rewards from language in context. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 8546–8560 (2022).
24. Sanh, V. et al. Multitask prompted training enables zero-shot task generalization. In *International Conference on Learning Representations* (2022).
25. Wei, J. et al. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations* (2022).
26. OpenAI. Introducing GPT-4.1 in the API. <https://openai.com/index/gpt-4-1/> (2025).
27. Chiang, W. L. et al. Chatbot Arena: an open platform for evaluating LLMs by human preference. In *International Conference on Machine Learning* (2024).
28. Llama Team. The llama 3 herd of models. Preprint at <https://arxiv.org/abs/2407.21783> (2024).
29. Yang, A. et al. Qwen2.5 technical report. Preprint at <https://arxiv.org/abs/2412.15115> (2024).
30. Wei, J. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems* 35 (2022).
31. Nye, M. et al. Show your work: Scratchpads for intermediate computation with language models. In *ICLR 2022 Workshop on Deep Learning for Code* (2022).
32. Kojima, T., Gu, S. S., Reid, M., Matsuo, Y. & Iwasawa, Y. Large language models are zero-shot reasoners. In *Advances in Neural Information Processing Systems* 35 (2022).
33. Rafailov, R. et al. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems* 36 (2023).
34. Hu, E. J. et al. LoRA: low-rank adaptation of large language models. In *International Conference on Learning Representations* (2022).
35. Hinton, G., Vinyals, O. & Dean, J. Distilling the knowledge in a neural network. In *NIPS 2014 Workshop on Deep Learning and Representation Learning* (2014).
36. Kim, Y. & Rush, A. M. Sequence-level knowledge distillation. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, 1317–1327 (2016).
37. Deng, Y. et al. Implicit chain of thought reasoning via knowledge distillation. Preprint at *arXiv* <https://arxiv.org/abs/2311.01460> (2023).
38. Wang, P. et al. SCOTT: self-consistent chain-of-thought distillation. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 5546–5558 (2023).
39. Li, L. H. et al. Symbolic chain-of-thought distillation: Small models can also “think” step-by-step. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2665–2679 (2023).
40. Jung, J. et al. Impossible distillation for paraphrasing and summarization: How to make high-quality lemonade out of small, low-quality models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 4439–4454 (2024).
41. Yu, P., Xu, J., Weston, J. E. & Kulikov, I. Distilling system 2 into system 1. In *The First Workshop on System-2 Reasoning at Scale, NeurIPS’24* (2024).
42. Chen, X. et al. Learning to maximize mutual information for chain-of-thought distillation. In *Findings of the Association for Computational Linguistics: ACL 2024*, 6857–6868 (2024).
43. Yao, S., Chen, H., Yang, J. & Narasimhan, K. Webshop: Towards scalable real-world web interaction with grounded language agents. In *Advances in Neural Information Processing Systems* 35 (2022).
44. Nafar, A., Venable, K. B. & Kordjamshidi, P. Reasoning over uncertain text by generative large language models. In *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence*, 24911–24920 (2025).
45. Paruchuri, A. et al. What are the odds? Language models are capable of probabilistic reasoning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 11712–11733 (2024).
46. Zhu, J.-Q. & Griffiths, T. L. Incoherent probability judgments in large language models. In *Proceedings of the 46th Annual Conference of the Cognitive Science Society* (2024).
47. Belem, C. G., Kelly, M., Steyvers, M., Singh, S. & Smyth, P. Perceptions of linguistic uncertainty by language models and humans. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 8467–8502 (2024).
48. Li, B. Z., Tamkin, A., Goodman, N. D. & Andreas, J. Eliciting human preferences with language models. In *International Conference on Learning Representations* (2025).
49. Handa, K. et al. Bayesian preference elicitation with language models. Preprint at *arXiv* <https://arxiv.org/abs/2403.05534> (2024).
50. Piriyaikulij, T., Kuleshov, V. & Ellis, K. Active preference inference using language models and probabilistic reasoning. In *NeurIPS 2023 Workshop on Foundation Models for Decision Making* (2023).
51. Andukuri, C., Fränken, J. P., Gerstenberg, T. & Goodman, N. D. STaR-GATE: Teaching language models to ask clarifying questions. In *Conference on Language Modeling* (2024).
52. Peng, A., Sun, Y., Shu, T. & Abel, D. Pragmatic feature preferences: Learning reward-relevant preferences from human input. In *International Conference on Machine Learning* (2024).
53. Aliannejadi, M., Kiseleva, J., Chuklin, A., Dalton, J. & Burtsev, M. Building and evaluating open-domain dialogue corpora with clarifying questions. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, 4473–4484 (2021).
54. Chen, S., Wiseman, S. & Dhingra, B. Chatshop: Interactive information seeking with language agents. Preprint at *arXiv* <https://arxiv.org/abs/2404.09911> (2024).
55. Zhao, S., Hong, M., Liu, Y., Hazarika, D. & Lin, K. Do LLMs recognize your preferences? evaluating personalized preference following in LLMs. In *International Conference on Learning Representations* (2025).
56. Feng, Y., Zhou, B., Lin, W. & Roth, D. BIRD: A trustworthy Bayesian inference framework for large language models. In *International Conference on Learning Representations* (2025).
57. Liu, R., Geng, J., Peterson, J. C., Sucholutsky, I. & Griffiths, T. L. Large language models assume people are more rational than we really are. In *International Conference on Learning Representations* (2025).
58. Piriyaikulij, T., Langenfeld, C., Le, T. A. & Ellis, K. Doing experiments and revising rules with natural language and probabilistic

- reasoning. In *Advances in Neural Information Processing Systems* 37 (2024).
59. Grand, G., Pepe, V., Andreas, J. & Tenenbaum, J. B. Loose lips sink ships: Asking questions in battleship with language-informed program sampling. In *Proceedings of the Annual Meeting of the Cognitive Science Society* (2024).
 60. Ying, L., Zhi-Xuan, T., Wong, L., Mansinghka, V. & Tenenbaum, J. Grounding language about belief in a Bayesian theory-of-mind. In *Proceedings of the Annual Meeting of the Cognitive Science Society* (2024).
 61. Ellis, K. Human-like few-shot learning via Bayesian reasoning over natural language. In *Advances in Neural Information Processing Systems* 36 (2023).
 62. Lakretz, Y. et al. The emergence of number and syntax units in LSTM language models. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 11–20 (2019).
 63. Hao, S. & Linzen, T. Verb conjugation in transformers is determined by linear encodings of subject number. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, 4531–4539 (2023).
 64. Manning, C. D., Clark, K., Hewitt, J., Khandelwal, U. & Levy, O. Emergent linguistic structure in artificial neural networks trained by self-supervision. In *Proceedings of the National Academy of Sciences* **117**, 30046–30054 (2020).
 65. Finlayson, M. et al. Causal analysis of syntactic agreement mechanisms in neural language models. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, 1828–1843 (2021).
 66. Ravfogel, S., Prasad, G., Linzen, T. & Goldberg, Y. Counterfactual interventions reveal the causal effect of relative clause representations on agreement prediction. In *Proceedings of the 25th Conference on Computational Natural Language Learning*, 194–209 (2021).
 67. Vig, J. et al. Investigating gender bias in language models using causal mediation analysis. In *Advances in Neural Information Processing Systems* 33 (2020).
 68. Wang, K., Variengien, A., Conmy, A., Shlegeris, B. & Steinhardt, J. Interpretability in the wild: a circuit for indirect object identification in GPT-2 small. In *International Conference on Learning Representations* (2023).
 69. Eisape, T. et al. A systematic comparison of syllogistic reasoning in humans and language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 8425–8444 (2024).
 70. Tversky, A. & Kahneman, D. Judgment under uncertainty: Heuristics and biases: Biases in judgments reveal some heuristics of thinking under uncertainty. *Science* **185**, 1124–1131 (1974).
 71. Simon, H. A. A behavioral model of rational choice. *The Quarterly Journal of Economics*, 99–118, (1955).
 72. Koehler, D. J. & James, G. Probability matching and strategy availability. *Mem. Cogn.* **38**, 667–676 (2010).
 73. Vaswani, A. et al. Attention is all you need. In *Advances in Neural Information Processing Systems* 30 (2017).
 74. Palan, S. & Schitter, C. Prolific.ac—A subject pool for online experiments. *Journal of Behavioral and Experimental Finance* **17**, 22–27 (2018).

Acknowledgments

L.Q. and Y.K. were supported by funds from the MIT-Google Program for Computing Innovation. We thank Stephanie Chan, Andrew Lampinen, Michael Mozer, Peter Shaw, and Zhaofeng Wu for helpful discussions.

Author contributions

L.Q., F.S., T.L., and S.V.S. co-led the project. S.V.S. conceptualized the project direction. L.Q. conducted the experiments and analysis. L.Q., F.S., T.L., and S.V.S. framed, analyzed and designed experiments, with inputs from K.A. and Y.K. L.Q., T.L., and S.V.S. wrote the paper with help from F.S., K.A., and Y.K. The majority of this work was done while L.Q. was a student researcher at Google.

Competing interests

The authors declare no competing interests.

Additional information

Supplementary information The online version contains supplementary material available at <https://doi.org/10.1038/s41467-025-67998-6>.

Correspondence and requests for materials should be addressed to Linlu Qiu, Tal Linzen or Sjoerd van Steenkiste.

Peer review information *Nature Communications* thanks Nicolás Marchant, Taylor Sorensen and Yi Xia for their contribution to the peer review of this work. A peer review file is available.

Reprints and permissions information is available at <http://www.nature.com/reprints>

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2026